Nama : Riri Fauzia Haris

Nim : 24250007

Prodi : Sistem Informasi

PELATIHAN SERTIFIKASI BNSP DAY-6

1. **create schema** akademik;

   create table akademik.mahasiswa(

   id **INTEGER**,

   nim **VARCHAR**(40),

   nama **VARCHAR**(50),

   jurusan **VARCHAR**(30),

   tanggal_lahir **DATE**

   );

   Membuat schema dengan nama akademik terlebih dahulu. Schema ini bisa dibilang sebagai tempat untuk mengelompokkan tabel agar database lebih terstruktur.
   Setelah itu, membuat tabel bernama mahasiswa di dalam schema akademik. Di dalam tabel tersebut terdapat lima kolom, yaitu id, nim, nama, jurusan, dan tanggal lahir. Masing-masing kolom juga memiliki tipe data sesuai dengan jenis informasi yang akan disimpan.

2. **insert into** akademik.mahasiswa(

   id, nim, nama, jurusan, tanggal_lahir

) **values** (

   1, '24250007', 'Riri Fauzia Haris', 'Sistem Informasi', '2005-04-12');

   Mulai memasukkan data mahasiswa pertama ke dalam tabel mahasiswa. Data yang dimasukkan berupa id 1, NIM 24250007, nama Riri Fauzia Haris, jurusan Sistem Informasi, dan tanggal lahir 12 April 2005.
   Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel.

3. **select * from** akademik.mahasiswa

   Menggunakan perintah SELECT * untuk menampilkan seluruh data yang ada di dalam tabel mahasiswa. Tanda bintang (*) artinya semua kolom akan ditampilkan, mulai dari id, nim, nama, jurusan, sampai tanggal lahir.

4. **insert into** akademik.mahasiswa(

   id, nim, nama, jurusan, tanggal_lahir

) **values** (

   2, '24250008', 'Paramadina Mulya', 'Sistem Informasi', '2006-03-13');

   Menggunakan perintah INSERT INTO untuk menambahkan data mahasiswa kedua. Data yang dimasukkan adalah ID 2, NIM 24250008, nama Paramadina Mulya, jurusan Sistem Informasi, dan tanggal lahir 13 Maret 2006.

5. **select * from** akademik.mahasiswa
   Menampilkan kembali seluruh data mahasiswa

6. **UPDATE** akademik.mahasiswa

   SET tanggal_lahir = '2006-04-19'

   WHERE nim = '24250007';

   Menggunakan perintah UPDATE untuk mengubah data tanggal lahir mahasiswa. Data yang diubah adalah mahasiswa dengan nim 24250007. Tanggal lahir yang sebelumnya 12 April 2005 diubah menjadi 19 April 2006.
   Perintah WHERE digunakan supaya perubahan hanya berlaku pada mahasiswa yang memiliki NIM tersebut, bukan mengubah semua data mahasiswa.

7. **select * from** akademik.mahasiswa
   Menampilkan kembali seluruh data mahasiswa

8. **UPDATE** akademik.mahasiswa

   SET tanggal_lahir = '2006-02-13'

   WHERE nim = '24250008';

Menggunakan perintah UPDATE, tetapi kali ini data yang diubah adalah tanggal lahir mahasiswa kedua dengan NIM 24250008.

Tanggal lahir yang sebelumnya 13 Maret 2006 diubah menjadi 13 Februari 2006. Perubahan ini hanya berlaku untuk mahasiswa yang memiliki NIM tersebut.

9. **select * from** akademik.mahasiswa
   Menampilkan kembali seluruh data mahasiswa

10. **INSERT INTO** akademik.mahasiswa (
    id, nim, nama, jurusan, tanggal_lahir
) **VALUES** (
    3, '25260023', 'Ananda Fitri', 'Teknik Informatika', '2006-10-26'
    );

    Menambahkan data mahasiswa ketiga ke dalam tabel mahasiswa. Data yang dimasukkan adalah ID 3, NIM 25260023, nama Ananda Fitri, jurusan Teknik Informatika, dan tanggal lahir 26 Oktober 2006.

    Jadi, jumlah data mahasiswa yang sebelumnya dua sekarang bertambah menjadi tiga.

11. **select * from** akademik.mahasiswa
    Menampilkan kembali seluruh data mahasiswa

12. **INSERT INTO** akademik.mahasiswa (
    id, nim, nama, jurusan, tanggal_lahir
) **VALUES**
    (4, '24260019', 'Ela Santia', 'Teknik Informatika', '2006-10-05'),
    (5, '24260038', 'Weri Yunelda', 'Teknik Informatika', '2006-01-10')

    Menambahkan dua data mahasiswa sekaligus dalam satu perintah INSERT INTO. Data pertama adalah Ela Santia dengan ID 4, sedangkan data kedua adalah Weri Yunelda dengan id 5. Keduanya berasal dari jurusan Teknik Informatika.

Dengan cara ini, tidak perlu membuat perintah INSERT secara terpisah untuk setiap mahasiswa

13. **select** * **from** akademik.mahasiswa
    Menampilkan kembali seluruh data mahasiswa

14. **select** * **from** akademik.mahasiswa **order BY** id **asc**;

    Menggunakan ORDER BY id ASC untuk mengurutkan data mahasiswa berdasarkan ID dari angka terkecil ke angka terbesar.

15. **INSERT INTO** akademik.mahasiswa (

    id, nim, nama, jurusan, tanggal_lahir

) **VALUES**

    (6, '24250021', 'Dimas Pratama', 'Sistem Informasi', '2005-08-17');

    Menambahkan satu data mahasiswa baru bernama Dimas Pratama dengan ID 6 dan NIM 24250021.

    Mahasiswa tersebut berasal dari jurusan Sistem Informasi dan memiliki tanggal lahir 17 Agustus 2005. Setelah perintah ini dijalankan, jumlah data mahasiswa bertambah menjadi enam.

16. **select** * **from** akademik.mahasiswa
    Menampilkan kembali seluruh data mahasiswa

17. **delete from** akademik.mahasiswa **where** id = 6;

    Menggunakan perintah `DELETE` untuk menghapus data mahasiswa yang memiliki ID 6, yaitu Dimas Pratama.

    Perintah `WHERE id = 6` digunakan supaya hanya data dengan ID tersebut yang dihapus. Jadi, data mahasiswa lainnya tetap aman dan tidak ikut terhapus.

18. **select** * **from** akademik.mahasiswa
    Menampilkan kembali seluruh data mahasiswa

19. **alter table** akademik.mahasiswa **add primary key** (id);

    Menggunakan perintah ALTER TABLE untuk mengubah struktur tabel mahasiswa dengan menambahkan Primary Key pada kolom ID.

Primary Key berfungsi sebagai identitas unik untuk setiap data mahasiswa. Artinya, setiap mahasiswa harus memiliki ID yang berbeda dan tidak boleh ada ID yang sama atau kosong. Dengan adanya Primary Key, data mahasiswa menjadi lebih teratur dan tidak mudah mengalami duplikasi pada bagian ID.

20. **select * from** akademik.mahasiswa **order by** id **desc**;

    Menggunakan ORDER BY id DESC untuk menampilkan data mahasiswa berdasarkan ID dari angka terbesar ke angka terkecil.

21. **select * from** akademik.mahasiswa **order BY** id **asc**;

    Mengurutkan data mahasiswa berdasarkan ID menggunakan perintah ASC

22. **SELECT * FROM** akademik.mahasiswa

    WHERE id **IN** (1, 3);

    Menggunakan kondisi WHERE IN untuk menampilkan data mahasiswa yang memiliki ID 1 dan ID 3 saja.

    Jadi, dari seluruh data mahasiswa yang ada, PostgreSQL hanya akan menampilkan dua data yang ID-nya sesuai dengan angka di dalam tanda kurung, yaitu 1 dan 3

23. **SELECT * FROM** akademik.mahasiswa

    WHERE jurusan = 'Teknik Informatika';

    Menggunakan perintah WHERE untuk mencari dan menampilkan mahasiswa yang berasal dari jurusan Teknik Informatika saja.

24. **select * from** akademik.mahasiswa **where** id = 2 **or** jurusan = 'Sistem Informasi';

    Menggunakan operator OR untuk menampilkan data mahasiswa berdasarkan dua kondisi.

    Kondisi pertama adalah mahasiswa yang memiliki id 2, sedangkan kondisi kedua adalah mahasiswa yang berasal dari jurusan Sistem Informasi.

    Operator OR artinya salah satu kondisi saja sudah cukup untuk menampilkan data. Jadi, mahasiswa akan tetap ditampilkan jika memenuhi salah satu atau kedua kondisi tersebut.

25. **SELECT * FROM** akademik.mahasiswa **WHERE EXTRACT**(**MONTH FROM** tanggal_lahir) > 6;

    Menggunakan fungsi EXTRACT untuk mengambil bagian bulan dari kolom tanggal lahir mahasiswa. Dan menggunakan kondisi `> 6`, yang artinya hanya menampilkan mahasiswa yang lahir pada bulan setelah bulan Juni.

26. **select * from** akademik.mahasiswa **where** nama **like** '%ri%';

    Menggunakan perintah `LIKE` untuk mencari data mahasiswa berdasarkan pola huruf yang terdapat pada nama.
    Pola `'%ri%'` artinya mencari nama yang mengandung huruf "ri", baik di bagian awal, tengah, maupun akhir nama.
    Tanda `%` digunakan sebagai pengganti karakter apa pun. Jadi, nama mahasiswa yang mengandung kombinasi huruf "ri" akan ditampilkan.

27. **INSERT INTO** akademik.mahasiswa (

    id, nim, nama, jurusan, tanggal_lahir

) **VALUES**

    (6, '24240008', 'Ahmad Fajari', 'Teknologi Agroindustri', '1999-08-07');

    Menambahkan data mahasiswa baru bernama Ahmad Fajari dengan ID 6, NIM 24240008, jurusan Teknologi Agroindustri, dan tanggal lahir 7 Agustus 1999.
    Setelah data dimasukkan, saya menggunakan perintah SELECT * untuk langsung menampilkan seluruh isi tabel mahasiswa dan memastikan bahwa data Ahmad Fajari sudah berhasil ditambahkan.

28. **select * from** akademik.mahasiswa
    Menampilkan kembali seluruh data mahasiswa

29. **CREATE table** akademik.produk (

    kode_produk **VARCHAR**(10) **PRIMARY KEY**,

    nama_produk **VARCHAR**(50) **NOT NULL**,

kategori **VARCHAR**(30),

harga **INT**,

stok **INT**

);

Membuat tabel baru bernama produk di dalam schema akademik. Tabel ini digunakan untuk menyimpan data barang atau produk. Di dalam tabel tersebut terdapat lima kolom, yaitu : Kode produk, nama produk,kategori, harga dan stok

30. **INSERT INTO** akademik.produk

(kode_produk, nama_produk, kategori, harga, stok)

VALUES

('PRD-001', 'Madu Hutan', 'Makanan', 56800, 24),

('PRD-002', 'Map Plastik', 'Alat Tulis', 39000, 15),

('PRD-003', 'Topi Baseball', 'Pakaian', 285300, 174),

('PRD-004', 'Buku Tulis', 'Alat Tulis', 52900, 80),

('PRD-005', 'Kaos Polos', 'Pakaian', 214600, 76),

('PRD-006', 'Tempat Sampah', 'Rumah Tangga', 193200, 199),

('PRD-007', 'Kemeja Flanel', 'Pakaian', 182400, 134);

Mengisi tabel produk dengan tujuh data produk sekaligus menggunakan perintah INSERT INTO.

Data yang dimasukkan terdiri dari Madu Hutan, Map Plastik, Topi Baseball, Buku Tulis, Kaos Polos, Tempat Sampah, dan Kemeja Flanel.

Setiap produk memiliki kode produk, nama, kategori, harga, dan jumlah stok masing-masing.

31. **select * from** produk

Menampilkan Seluruh Data Produk

32. **select * from** produk **where** stok < 50;

Menggunakan kondisi WHERE stok < 50 untuk menampilkan produk yang jumlah stoknya kurang dari 50.

33. **truncate** akademik.produk

Menggunakan perintah TRUNCATE untuk menghapus seluruh data yang ada di dalam tabel produk.

Berbeda dengan DELETE yang bisa digunakan untuk menghapus data berdasarkan kondisi tertentu, TRUNCATE akan mengosongkan seluruh isi tabel sekaligus tanpa menghapus struktur tabelnya